Clustering with K-means

🎯 Today's goal

By the end of this hour you will:

- know what **unsupervised learning** is: learning without any right answers
- understand the four steps of **K-means**: pick centres, assign, move, repeat
- use `KMeans` and read `labels_`, `cluster_centers_` and `inertia_`
- choose the number of clusters with the **elbow method**
- **scale** your data first, and describe customer segments in plain words

🤔 Why this matters

Pour a big bag of mixed sweets on a table and ask a friend to sort them. Nobody tells them the "right" groups, yet they will make piles: by colour, by size, by wrapper. They put **similar things together**.

Since Day 16 every model learned from examples *with* the right answer (the label `y`). That is **supervised** learning. Often there is no label at all: a shop has thousands of customers but nobody has written "bargain hunter" or "big spender" next to them. **Clustering** finds such groups by itself. This is **unsupervised** learning. Shops use it for customer segments, biologists to group cells, and journalists to group similar documents.

🫧 Data with groups in it

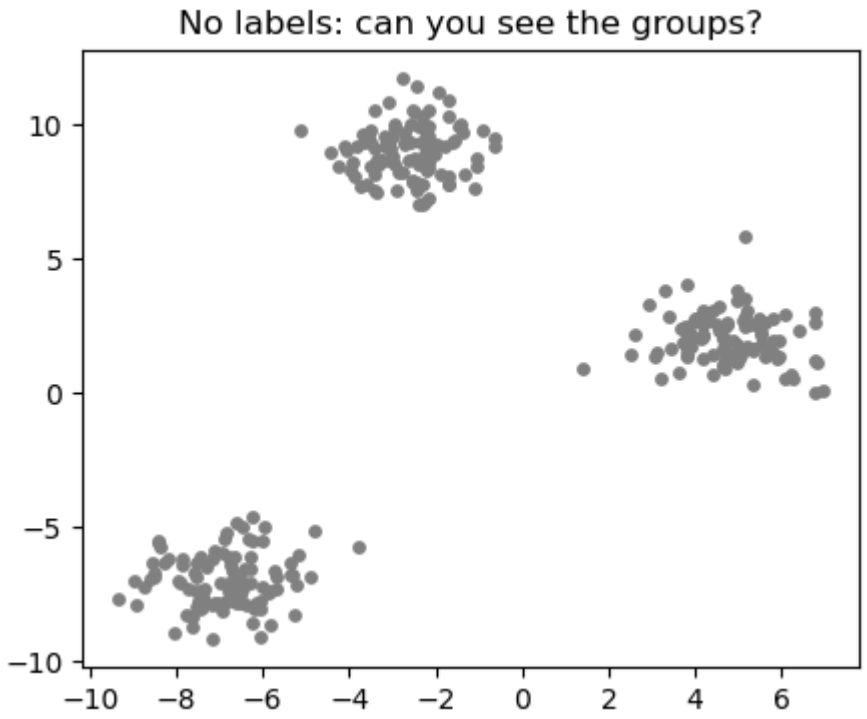
scikit-learn's `make_blobs` makes up points that sit in round "blobs". It is perfect for seeing how clustering works. It also returns the true blob of each point, but we throw that away with `_` (a name that means "I don't need this"): we pretend we don't know the groups.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_blobs

# 300 made-up points with 2 features, in 3 hidden groups
X_blobs, _ = make_blobs(n_samples=300, centers=3, cluster_std=1.0, random_state=42)
print("Shape:", X_blobs.shape)

plt.figure(figsize=(5, 4))
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], color="gray", s=15)
plt.title("No labels: can you see the groups?")
plt.show()
```

```
Shape: (300, 2)
```



Your eyes find three groups at once. Now let's teach the computer to do the same.

👣 How K-means works

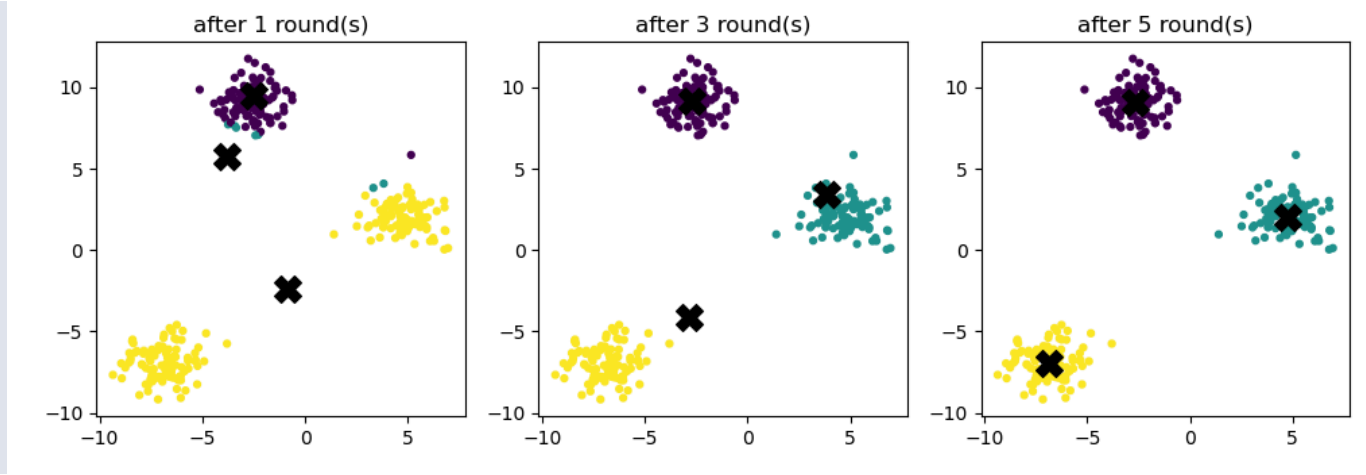
K is the number of groups you ask for, and a **mean** is an average. K-means describes every group by its **centre** (the average point of the group) and repeats four simple steps:

1. **Pick** K starting centres at random.
2. **Assign** every point to its nearest centre.
3. **Move** each centre to the average of the points assigned to it.
4. **Repeat** steps 2 and 3 until the centres stop moving.

We can watch it happen. `max_iter` limits how many rounds of "assign + move" it may do, so we stop it after 1 round, after 3 rounds, and when it is finished. The black crosses are the centres:

```
from sklearn.cluster import KMeans

fig, axes = plt.subplots(1, 3, figsize=(12, 3.6))
for ax, rounds in zip(axes, [1, 3, 10]):
    km = KMeans(n_clusters=3, init="random", n_init=1,
                max_iter=rounds, random_state=4)
    km.fit(X_blobs)
    ax.scatter(X_blobs[:, 0], X_blobs[:, 1], c=km.labels_, cmap="viridis", s=12)
    centres = km.cluster_centers_
    ax.scatter(centres[:, 0], centres[:, 1], color="black", marker="X", s=200)
    ax.set_title(f"after {km.n_iter_} round(s)")
plt.show()
```



At first, two centres share one real group and one centre must cover two groups. Round by round the centres slide towards the middle of the real groups. (`init="random"`, `n_init=1` and `max_iter` are only used here to show the steps. Normally you leave them out.)

🤖 K-means in scikit-learn

The usual pattern, but notice: `fit` gets only `X`. There is no `y`, because there are no answers. Afterwards the model holds:

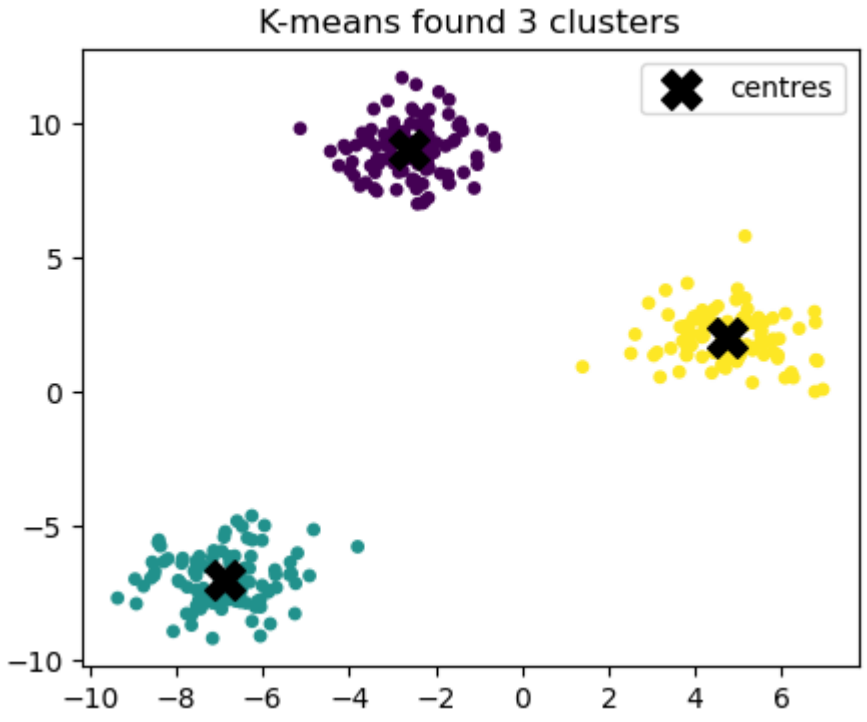
- `labels_`: the cluster number of every point (0, 1, 2…)
- `cluster_centers_`: the coordinates of each centre

```
kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
kmeans.fit(X_blobs)

print("First 10 labels:", kmeans.labels_[:10])
print("Points per cluster:", np.bincount(kmeans.labels_))
print("Centres:")
print(kmeans.cluster_centers_.round(2))

plt.figure(figsize=(5, 4))
plt.scatter(X_blobs[:, 0], X_blobs[:, 1], c=kmeans.labels_, cmap="viridis", s=15)
plt.scatter(kmeans.cluster_centers_[:, 0], kmeans.cluster_centers_[:, 1],
            color="black", marker="X", s=200, label="centres")
plt.title("K-means found 3 clusters")
plt.legend()
plt.show()
```

```
First 10 labels: [1 1 0 2 1 2 0 2 0 0]
Points per cluster: [100 100 100]
Centres:
[[-2.63  9.04]
 [-6.88 -6.98]
 [ 4.75  2.01]]
```



New tool: `np.bincount(labels)` counts how many times each number 0, 1, 2... appears.

The numbers are just names. "Cluster 0" is not better or first in any way. Run with a different `random_state` and the same groups may get different numbers. It's up to you, the human, to look at each cluster and give it a meaningful name.

A new point simply joins the cluster with the nearest centre. `predict` does that:

```
new_points = [[-3, 9], [5, 2], [-7, -7]]
kmeans.predict(new_points)
```

```
array([0, 2, 1])
```

📏 How tight are the clusters? Inertia

`inertia_` adds up, for every point, the **squared distance** to its own centre. Small inertia means the points sit close to their centres: tight clusters. K-means always tries to make this number as small as possible.

Because K-means starts from *random* centres, an unlucky start can get stuck in a poor answer. Let's try 5 single starts:

```
print(f"Our model (n_init=10): inertia = {kmeans.inertia_:.0f}")

for seed in [0, 1, 2, 3, 4]:
    one_try = KMeans(n_clusters=3, init="random", n_init=1, random_state=seed)
    one_try.fit(X_blobs)
    print(f"single random start {seed}: inertia = {one_try.inertia_:.0f}")
```

```
Our model (n_init=10): inertia = 567
single random start 0: inertia = 5695
single random start 1: inertia = 5693
single random start 2: inertia = 567
single random start 3: inertia = 567
single random start 4: inertia = 567
```

Some starts got stuck with a much bigger inertia (two centres shared one blob). That is why we write `n_init=10`: K-means runs 10 times from different starts and keeps the run with the smallest inertia. Together with `random_state=42`, the result is good *and* the same every time.

💪 How many clusters? The elbow method

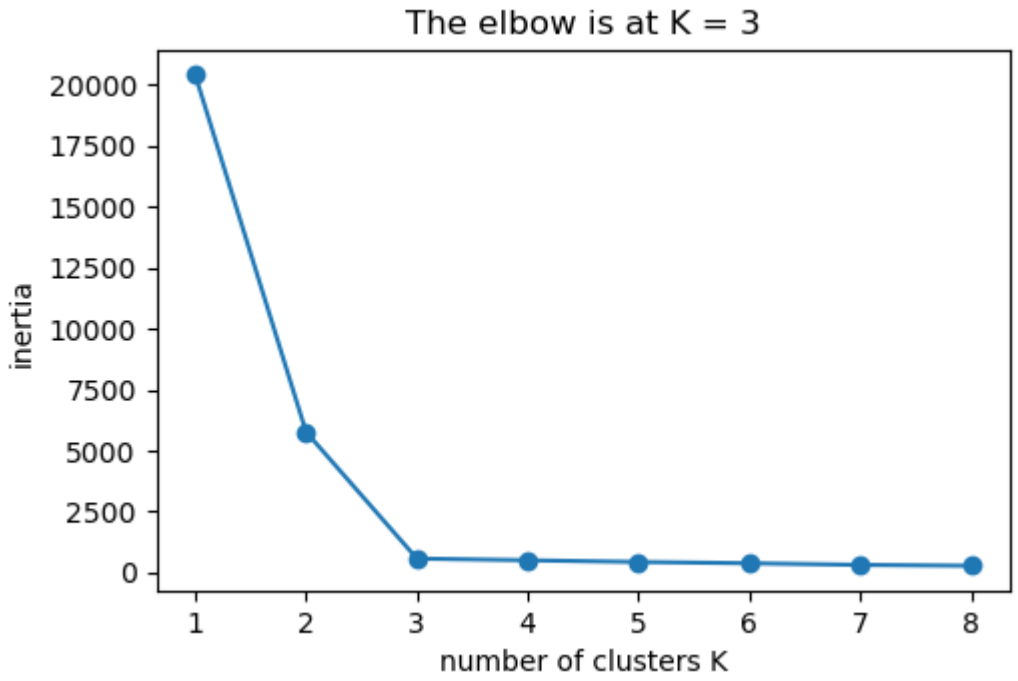
With real data you don't know K. Could we just pick the K with the smallest inertia? No: more clusters *always* means smaller inertia (with one cluster per point it would be 0!). Instead we draw inertia for K = 1, 2, 3… and look for the **elbow**: the point where adding another cluster stops helping much.

```
ks = range(1, 9)
inertias = []
for k in ks:
    model = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_blobs)
    inertias.append(model.inertia_)

print([round(value) for value in inertias])

plt.figure(figsize=(5.5, 3.5))
plt.plot(list(ks), inertias, marker="o")
plt.xlabel("number of clusters K")
plt.ylabel("inertia")
plt.title("The elbow is at K = 3")
plt.show()
```

```
[20402, 5763, 567, 496, 427, 375, 308, 272]
```



From 1 to 2 to 3 clusters the inertia falls steeply; after 3 the line is almost flat. The bend, like an elbow in an arm, says **K = 3**. Real data rarely has such a sharp elbow, so also ask: "Can I describe and use these groups?"

🛍️ Customer segments, and why to scale first

Now a **made-up** shop with 120 customers. For each one we know how much they spent last year (in dollars) and how many times a month they visit:

```
rng = np.random.default_rng(42)
n = 40     # 40 customers in each hidden group (made-up data)
spend = np.concatenate([rng.normal(1000, 250, n),      # group A
                        rng.normal(1000, 250, n),      # group B
                        rng.normal(3000, 300, n)])     # group C
visits = np.concatenate([rng.normal(2, 0.8, n),
                         rng.normal(12, 1.5, n),
                         rng.normal(7, 1.5, n)])
spend = spend.round(0)
visits = visits.clip(1).round(0)     # at least 1 visit, whole numbers

customers = pd.DataFrame({
    "customer_id": [f"C{number:03d}" for number in range(1, 3 * n + 1)],
    "spend": spend,
    "visits": visits,
})
customers.head()
```

	customer_id	spend	visits
0	C001	1076.0	1.0
1	C002	740.0	2.0
2	C003	1188.0	2.0
3	C004	1235.0	3.0
4	C005	512.0	3.0

`f"C{number:03d}"` writes a number with 3 digits, padded with zeros: C001, C002...

K-means measures distances, so it only understands numbers. Giving it the whole table, ID text included, fails:

💡 This example has a mistake on purpose. Run it and read the error.

```
KMeans(n_clusters=3, n_init=10, random_state=42).fit(customers)
```

> **⚠️ ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 1, in <module>
>     KMeans(n_clusters=3, n_init=10, random_state=42).fit(customers)
>     ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^
> ValueError: could not convert string to float: 'C001'
> ```

"could not convert string to float: 'C001'": Python tried to turn the ID text into a number. Select only the number columns: `customers[["spend", "visits"]]`.

There's a second, sneakier problem. Spend is in the **thousands**, visits are below **20**. A difference of 10 visits (a huge change in behaviour!) counts as much as a difference of 10 dollars (nothing). So K-means looks almost only at spend. Compare the clusters without and with scaling. `StandardScaler` (Day 18) puts both columns on the same scale: mean 0, standard deviation 1.

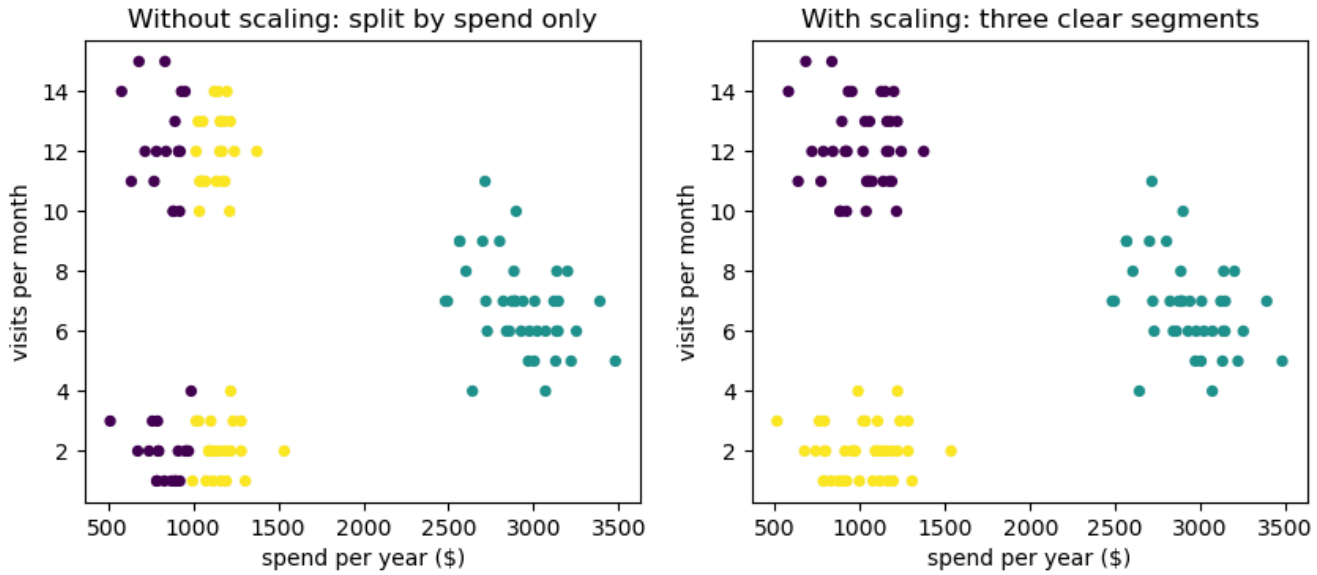
```
from sklearn.preprocessing import StandardScaler

features = customers[["spend", "visits"]]

raw_labels = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(features)

scaled = StandardScaler().fit_transform(features)
scaled_labels = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(scaled)

fig, axes = plt.subplots(1, 2, figsize=(10, 3.8))
axes[0].scatter(customers["spend"], customers["visits"], c=raw_labels, s=18)
axes[0].set_title("Without scaling: split by spend only")
axes[1].scatter(customers["spend"], customers["visits"], c=scaled_labels, s=18)
axes[1].set_title("With scaling: three clear segments")
for ax in axes:
    ax.set_xlabel("spend per year ($)")
    ax.set_ylabel("visits per month")
plt.show()
```



`fit_predict` is a shortcut: fit, then give back `labels_`, in one call.

Without scaling, the low spenders are cut into two halves by spend, ignoring that some of them visit 12 times a month and others twice. After scaling, both columns count, and the three real groups appear. **Rule: scale before clustering** whenever columns use different units.

Clusters are only useful once you can **describe** them. Add the labels as a column and compare the groups with `groupby` (Day 11):

```
customers["cluster"] = scaled_labels

summary = customers.groupby("cluster")[["spend", "visits"]].mean().round(1)
summary["customers"] = customers["cluster"].value_counts().sort_index()
summary
```

	spend	visits	customers
cluster			
0	1003.3	12.1	40
1	2930.2	6.8	40
2	1009.6	2.0	40

Now a human gives the segments names, for example: *"rare visitors"* (low spend, about 2 visits), *"frequent small shoppers"* (low spend, about 12 visits) and *"big spenders"* (high spend). The shop could send each group a different offer.

🌸 Does clustering find real groups?

On real data we can't check clusters, because there are no answers. But the iris flowers from Day 16 *do* have answers (the species). So let's cluster them while hiding the species, and only afterwards compare with `pd.crosstab` (Day 14):

```
from sklearn.datasets import load_iris

iris = load_iris(as_frame=True)
iris_kmeans = KMeans(n_clusters=3, n_init=10, random_state=42)
iris_kmeans.fit(iris.data)          # only X, no y!

species = iris.target.map({0: "setosa", 1: "versicolor", 2: "virginica"})
pd.crosstab(species, iris_kmeans.labels_,
            rownames=["real species"], colnames=["cluster"])
```

cluster	0	1	2
real species			
setosa	0	50	0
versicolor	48	0	2
virginica	14	0	36

Without ever seeing a species name, K-means puts all 50 setosa flowers in one cluster. Versicolor and virginica look more alike, so some of them are mixed up. Clustering finds groups that are *similar*, which is not always the same as the groups we care about.

⚠️ Common mistakes

- Giving K-means text columns (like IDs or names) → `ValueError: could not convert string to float`.
- Forgetting to scale columns with different units. The biggest numbers win.
- Thinking cluster numbers mean something. "Cluster 0" is just a name.
- Choosing K by the smallest inertia. More clusters always lower it; look for the elbow.
- Leaving out `n_init=10` and `random_state=42`: results can change, or get stuck in a poor answer.
- Believing clusters are "the truth". They are groups of similar points; a human must check that they make sense.

✅ Recap

- Unsupervised learning: only `X`, no `y`. Clustering puts similar points together.
- K-means: pick K centres → assign each point to the nearest → move centres to the average → repeat.

- `KMeans(n_clusters=K, n_init=10, random_state=42).fit(X)`, then `labels_`, `cluster_centers_`, `inertia_`, `predict(new)`.
- Elbow method: plot inertia for K = 1, 2, 3... and pick the bend.
- Scale first (`StandardScaler`), then describe each cluster with `groupby(...).mean()`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Fit K-means

The made-up points in `X` come in 4 blobs. Create a `KMeans` with **4** clusters, `n_init=10` and `random_state=42` in a variable called `kmeans`, fit it on `X`, and store its labels in `labels`.

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)

# Create and fit kmeans, then store labels
```

Hints:

1. `kmeans = KMeans(n_clusters=4, n_init=10, random_state=42)`
2. Fit with `kmeans.fit(X)`. Only `X`: there is no `y`.
3. `labels = kmeans.labels_`

2. Cluster sizes

The blobs have different sizes. Count how many points are in each cluster and store the counts in `sizes` (a pandas Series or a dict: cluster number → count). Then store the number of the **biggest** cluster in `biggest_cluster`, without typing it by hand.

```
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)
labels = kmeans.labels_

# Count sizes, then find biggest_cluster
```

Hints:

1. Turn the labels into a Series and count them: `pd.Series(labels).value_counts()`
2. For a tidy result, add `.sort_index()` so the clusters are in the order 0, 1, 2, 3.
3. The label with the biggest count: `biggest_cluster = sizes.idxmax()`

3. Elbow numbers

For every K from 1 to 6, fit a `KMeans(n_clusters=k, n_init=10, random_state=42)` on `X` and append its `inertia_` to the list `inertias`. Then look at the printed numbers (or draw them) and store the K at the elbow in `best_k`.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)

inertias = []
# Loop over k = 1 to 6 here

print([round(value) for value in inertias])
# best_k = ...
```

Hints:

1. `for k in range(1, 7):` gives 1, 2, 3, 4, 5, 6.
2. Inside: `model = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)`, then `inertias.append(model.inertia_)`
3. Find where the big drops stop: after that K, the numbers only shrink a little. The data was made with 4 blobs...

4. Which cluster do new points join?

Three new made-up points arrived. Use the fitted `kmeans` to find their clusters and store the result in `new_labels`. Then store the centre of the cluster that the **first** new point joined in `first_centre`.

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)

new_points = [[9, 1], [-8, 6], [0, -9]]

# Find new_labels, then first_centre
```

Hints:

1. `new_labels = kmeans.predict(new_points)`
2. The first point's cluster number is `new_labels[0]`.
3. The centres are rows of `kmeans.cluster_centers_`: `first_centre = kmeans.cluster_centers_[new_labels[0]]`

5. Challenge: customer segments

Cluster the made-up café customers below into **3 segments**:

1. Scale the `spend` and `visits` columns with `StandardScaler`.
2. Fit `KMeans(n_clusters=3, n_init=10, random_state=42)` on the scaled numbers.
3. Add the labels to the table as a new column `"cluster"`.
4. Store the mean `spend` and `visits` of each cluster in `summary` (use `groupby`).
5. Store the cluster number with the highest mean spend in `top_spenders`.

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

rng = np.random.default_rng(7)
n = 30     # made-up data: 30 customers per hidden group
spend = np.concatenate([rng.normal(200, 40, n), rng.normal(200, 40, n),
                        rng.normal(600, 60, n)])
visits = np.concatenate([rng.normal(3, 1, n), rng.normal(15, 2, n),
                         rng.normal(8, 2, n)])
cafe = pd.DataFrame({"spend": spend.round(0), "visits": visits.clip(1).round(0)})

# Your code here
```

Hints:

1. `scaled = StandardScaler().fit_transform(cafe[["spend", "visits"]])`
2. `cafe["cluster"] = KMeans(n_clusters=3, n_init=10, random_state=42).fit_predict(scaled)`
3. `summary = cafe.groupby("cluster")[["spend", "visits"]].mean()` and `top_spenders = summary["spend"].idxmax()`

➡️ Tomorrow

Mini-project: survival predictor. Remember the made-up *Northern Star* passengers from Day 15? Tomorrow you'll put the whole Machine Learning phase together: clean the data, turn text columns into numbers, train and compare models, evaluate them properly, and predict survival for new made-up passengers.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Fit K-means

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)

kmeans = KMeans(n_clusters=4, n_init=10, random_state=42)
kmeans.fit(X)
labels = kmeans.labels_
print("Points per cluster:", np.bincount(labels))
```

2. Cluster sizes

```
import pandas as pd
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)
labels = kmeans.labels_

sizes = pd.Series(labels).value_counts().sort_index()
biggest_cluster = sizes.idxmax()
print(sizes)
print("Biggest cluster:", biggest_cluster)
```

3. Elbow numbers

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)

inertias = []
for k in range(1, 7):
    model = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X)
    inertias.append(model.inertia_)

print([round(value) for value in inertias])
best_k = 4
```

4. Which cluster do new points join?

```
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

X, _ = make_blobs(n_samples=[80, 50, 40, 30], cluster_std=1.0, random_state=7)
kmeans = KMeans(n_clusters=4, n_init=10, random_state=42).fit(X)

new_points = [[9, 1], [-8, 6], [0, -9]]

new_labels = kmeans.predict(new_points)
first_centre = kmeans.cluster_centers_[new_labels[0]]
print("New labels:", new_labels)
print("Centre of the first point's cluster:", first_centre.round(2))
```

5. Challenge: customer segments

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

rng = np.random.default_rng(7)
n = 30     # made-up data: 30 customers per hidden group
spend = np.concatenate([rng.normal(200, 40, n), rng.normal(200, 40, n),
                        rng.normal(600, 60, n)])
visits = np.concatenate([rng.normal(3, 1, n), rng.normal(15, 2, n),
                         rng.normal(8, 2, n)])
cafe = pd.DataFrame({"spend": spend.round(0), "visits": visits.clip(1).round(0)})

scaled = StandardScaler().fit_transform(cafe[["spend", "visits"]])
model = KMeans(n_clusters=3, n_init=10, random_state=42)
cafe["cluster"] = model.fit_predict(scaled)

summary = cafe.groupby("cluster")[["spend", "visits"]].mean()
top_spenders = summary["spend"].idxmax()
print(summary.round(1))
print("Top spenders: cluster", top_spenders)
```